

MCP Servers



Tauhid Zaman

| What Is MCP?

- **Model Context Protocol** — a standard way for an AI agent to call tools and other documents that live in another program
- Someone sets up the server once with the tools and documents; any agent can connect to the server and use them
- Today, MCP functions as the standard connectivity layer for agentic AI, decoupling model intelligence from tools and environment context.



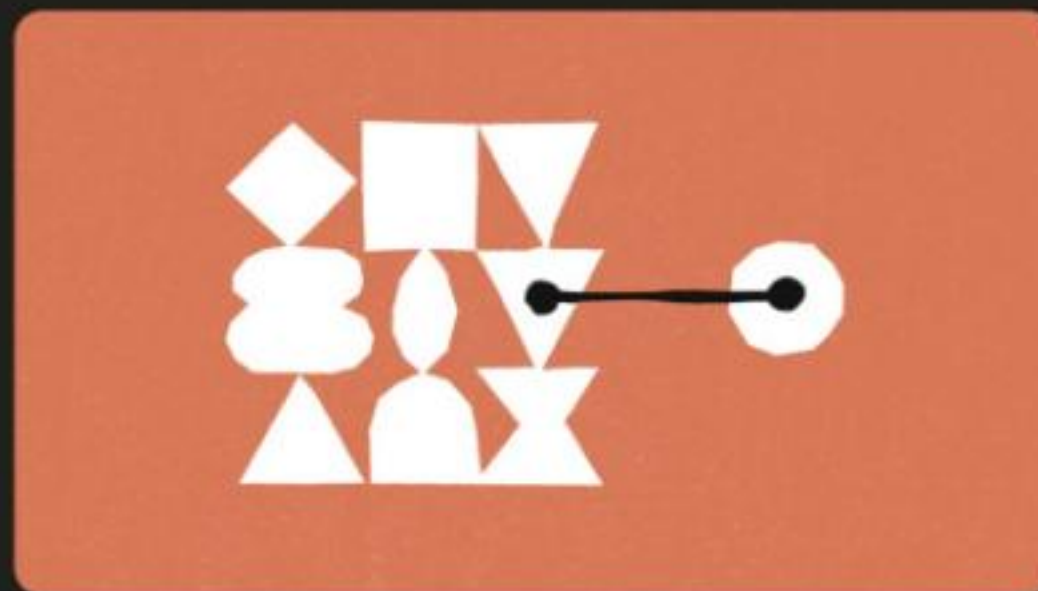
MCP Origins

- AI models were stuck in chat silos — every app needed a custom hookup to every data source. No single open standard existed
- To solve this problem, Anthropic open-sourced MCP on **November 25, 2024**
- **First users:** Claude Desktop at launch — Anthropic also shipped ready-made MCP servers for GitHub, Slack, Google Drive, and Postgres so people could try it immediately
- **Today: 10,000+** public MCP servers; all major vibe coders can use MCP servers

Sources: [Anthropic launch](#) · [10k+ servers](#) / [platform adoption](#)

Introducing the Model Context Protocol

Nov 25, 2024



Agents and MCP Servers

- The MCP server hands your agent a **tool belt** — a menu of what it can do
- **Tools** — actions it can run (search courses, read email, query a table)
- **Resources** — readable facts it can fetch (a rules file, a data sheet)
- Until it connects, the agent only has chat. After it connects, it can use those tools on real systems



MCP Request Flow

- Same loop as a normal agent with tools — ask, pick a tool, run it, get facts back
- The tools live on the **MCP server**; the agent calls them through that connection



MCP Examples

- **GitHub**

- Issues, pull requests, and repo search



- **Supabase**

- Query the cloud Postgres tables behind your Supabase project



- **Slack**

- Search channels, read threads, send messages



- **Outlook**

- Read or send mail through Microsoft Graph (Microsoft's API for Microsoft 365 — mail, calendar, OneDrive, Teams, etc.)



- **Spotify**

- Search songs, control playback, create playlists



MCP: Connect Or Build

Connect to an existing MCP Server

Use when a solid MCP server already exists

- GitHub, Slack, Supabase, Outlook, Spotify...
- Add the server in your agent and sign in if asked
- Easy — someone else maintains the tools



Build your own MCP server

Use when the system is yours

- Internal catalogue, shop inventory, company rules
- You write `mcp_server.py`, name the tools, and decide what each returns
- You control what the agent can see and what it must refuse



MCP In Python

- **FastMCP** — a Python package for building MCP servers; you name the server, then hang tools on it
- An MCP server is mostly tools — mark each one with `@mcp.tool`

mcp_server.py (sketch)

Copy

```
from fastmcp import FastMCP

mcp = FastMCP("yale-som-courses")

@mcp.tool
def search_courses(query: str) → dict:
    """Search the Yale SOM catalogue by keyword."""
    ...

@mcp.tool
def get_course(course_number: str) → dict:
    """Look up one course by number (e.g. MGT 409)."""
    ...

@mcp.tool
def list_courses_by_faculty(name: str) → dict:
    """List courses taught by a faculty member."""
    ...
```

What An MCP Server Can Expose

- Same `@mcp...` decorator pattern — three main kinds
 - **Tools** — Actions the agent can **call** — search, look up, write a row
 - **Resources** — Readable content the host can **fetch** — a rules file, a sheet of facts
 - **Prompts** — Named prompt **templates** the server ships — reusable instructions

@mcp.tool

Actions the agent can **call** — search, look up, write a row

example

Copy

```
@mcp.tool
def search_courses(query: str) → dict:
    """Search the catalogue."""
    ...
```

@mcp.resource

Readable content the host can **fetch** — a rules file, a sheet of facts

example

Copy

```
@mcp.resource("rules://desk")
def desk_rules() → str:
    return Path("rules.md").read_text()
```

@mcp.prompt

Named prompt **templates** the server ships — reusable instructions

example

```
@mcp.prompt()
def review_course() → str:
    return "Summarize this course for
```

Tools Vs MCP

- Same tool functions — `agent.py` wires them locally, or plugs into an MCP server

tools.py

Copy

```
def search_courses(query: str) → dict:
    """Search the catalogue by keyword."""
    ...

def get_course(course_number: str) → dict:
    """Look up one course by number."""
    ...

def list_courses_by_faculty(name: str) → dict:
    """List courses taught by a faculty member."""
    ...
```

mcp_server.py

Copy

```
@mcp.tool
def search_courses(query: str) → dict:
    """Search the catalogue by keyword."""
    ...

@mcp.tool
def get_course(course_number: str) → dict:
    """Look up one course by number."""
    ...

@mcp.tool
def list_courses_by_faculty(name: str) → dict:
    """List courses taught by a faculty member."""
    ...
```

agent.py

Copy

```
Agent(
    ...,
    tools=[search_courses, get_course, list_courses_by_faculty],
)
```

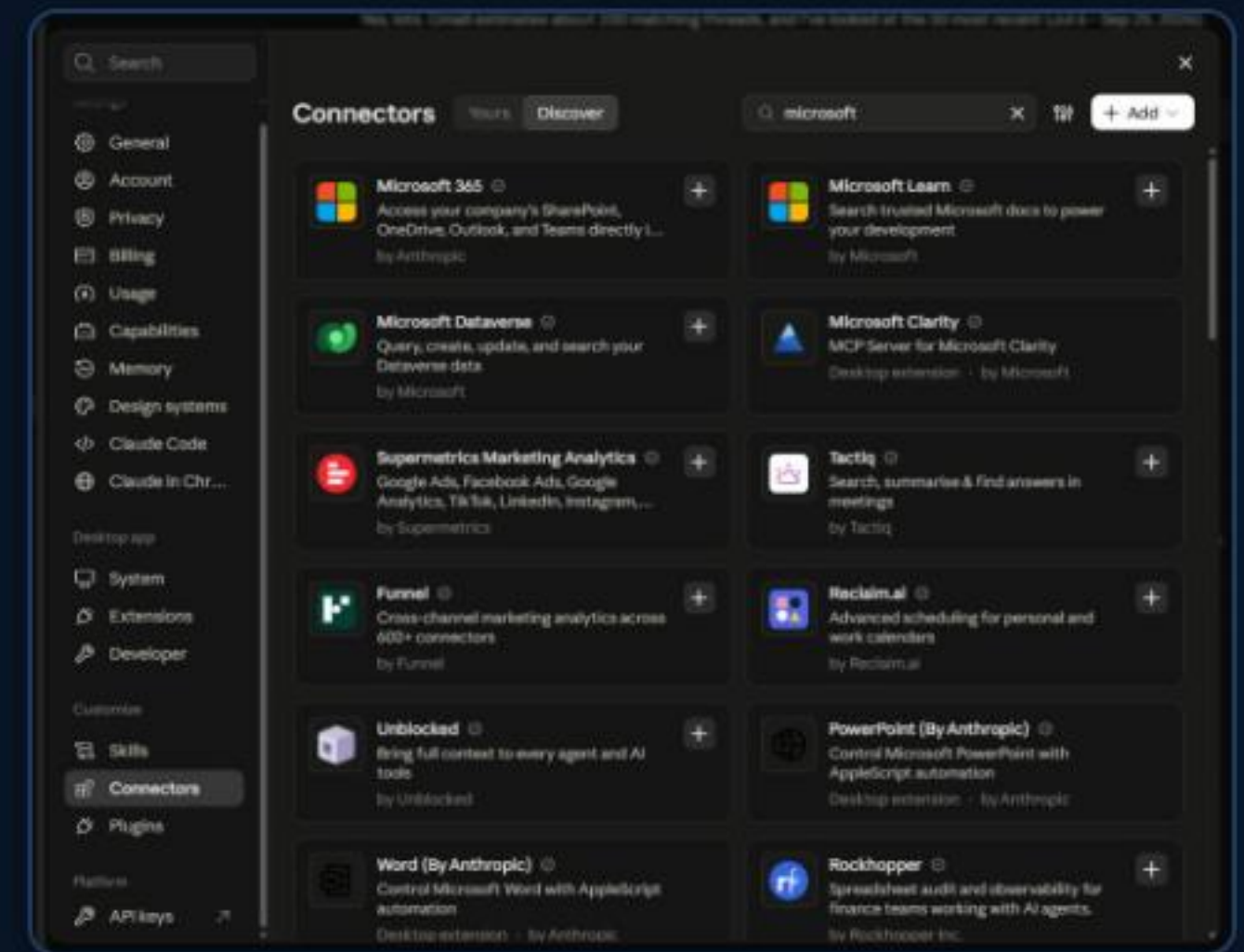
agent.py

Copy

```
Agent(
    ...,
    toolsets=[MCPToolset(mcp)],
)
```

Claude Code Connectors

- **Connectors** — Anthropic's name for plugging apps into Claude and Claude Code
- Under the hood, a connector is an **MCP server**
- Browse the Connector directory in Settings menu, click **+** to add a connector
- Sometimes your organization's IT blocks connectors for safety
- When Claude doesn't have the connector you need, you need to build your own MCP server



| Building Your Own Outlook MCP Server

1. Create an Azure app with the required mail permissions
2. Put the Application (client) ID in your `.env`
3. Write MCP server code
`mcp_outlook_server.py`
4. Run MCP server once, login, save tokens
5. Add MCP server to Claude Code (or whatever vibe coder you use)



1. Create An Azure App

- [Register an app](#) in Azure (click **+New registration**)
- **API permissions** → **Add a permission** → **Microsoft APIs** → **Microsoft Graph**(it will be big icon at top of page)
- Pick **Delegated permissions**
- Check `User.Read` and `Mail.Send` (Yale SOM IT only allows these now)
- Click **Add permissions**
- **Authentication (Preview)** → **Settings** → turn on **Allow public client flows**

Mail		
☐	Mail.Read ⓘ Read user mail	No
☐	Mail.Read.Shared ⓘ Read user and shared mail	No
☐	Mail.ReadBasic ⓘ Read user basic mail	No
☐	Mail.ReadBasic.Shared ⓘ Read user and shared basic mail	No
☐	Mail.ReadWrite ⓘ Read and write access to user mail	No
☐	Mail.ReadWrite.Shared ⓘ Read and write user and shared mail	No
☐	Mail.Send ⓘ Send mail as a user	No
☐	Mail.Send.Shared ⓘ Send mail on behalf of others	No

2. Put The Client ID In `.env`

- On the app Overview, copy **Application (client) ID** and put in your `.env` file
- In `.env`, Tenant can be `yale.edu`

`.env`

Copy

```
OUTLOOK_CLIENT_ID=your-application-client-id  
OUTLOOK_TENANT_ID=yale.edu
```

3. Write `mcp_outlook_server.py`

- FastMCP server named something like `outlook-mail`
- Wrap Graph calls with `@mcp.tool` — inbox, send, ...
- Load `OUTLOOK_CLIENT_ID` from `.env`
- End with `mcp.run()` for stdio
 - Claude launches your Python file and talks to it over the terminal pipes — no URL needed

`mcp_outlook_server.py` (sketch)

Copy

```
from fastmcp import FastMCP
mcp = FastMCP("outlook-mail")

@mcp.tool
def list_inbox(top: int = 10) → dict:
    """List recent inbox messages."""
    ...

@mcp.tool
def send_message(to: str, subject: str, body: str) → dict:
    """Send email as the signed-in user."""
    ...

if __name__ == "__main__":
    mcp.run()
```

4. Run Once, Login, Save Tokens

- Run `python mcp_outlook_server.py`
- Complete the device-code login in the browser
- Tokens save to `.token_cache.json` — you are not asked every time
- Keep `.env` and `.token_cache.json` off GitHub

terminal

Copy

```
python mcp_outlook_server.py
```

```
# First tool call → device-code URL in the terminal
```

```
# Sign in once → tokens land in .token_cache.json
```

5. Add MCP Server To Claude Code

- Update Claude Code's MCP list in its configuration JSON file (ask it how to do this)
- Put in your real path to `mcp_outlook_server.py` so Claude can run your server file
- Restart Claude → try listing inbox or sending a test to yourself

MCP config

Copy

```
{
  "mcpServers": {
    "outlook-mail": {
      "command": "python",
      "args": [
        "C:/Users/you/outlook_mcp/mcp_outlook_server.py"
      ]
    }
  }
}
```

| Using and Building MCP Servers

- You now know how MCP servers work — connect or build your own
- In our vibe coding session, we'll connect some MCP servers to our vibe coder
- Then we'll build a simple MCP server and plug it in
- [Vibe coding session](#) →

